

PROJECT_MAP

项目代码结构与函数调用说明

1. 总体架构

项目分为四层：

```
网页层 webapp/  
-> API 路由 app/main.py  
-> 业务服务 app/services.py  
-> 数据采集 spiders/ 与数据读写 app/repository.py  
-> SQLite 数据库 data/game_market.db 与 JSON 文件 data/raw、data/cleaned
```

核心业务是：

1. 定时或手动爬取三类数据源。
2. 对原始数据进行清洗、过滤脏数据、去重。
3. 将 cleaned 数据写入 SQLite 首页表。
4. 网页读取 API，展示热门游戏、B站热度、爬取状态。
5. 用户搜索游戏时，聚合基础信息、直播入口、攻略视频和诊断建议。

2. app 后端文件

app/main.py

FastAPI 应用入口。

主要对象：

- `app = FastAPI(...)`：创建 Web 应用。
- `crawl_service = CrawlService()`：创建爬取服务。
- `scheduler = create_scheduler(crawl_service)`：创建定时任务。

主要函数：

- `on_startup()`：启动时初始化数据库、启动定时器，并后台刷新一次数据。
- `on_shutdown()`：关闭服务时停止定时器。

- `index()`：返回首页模板 `webapp/templates/index.html`。
- `get_dashboard()`： `GET /api/dashboard`，返回首页图表和卡片数据。
- `batch_status()`： `GET /api/batch-status`，返回三个 cleaned JSON 文件的条数和更新时间。
- `search_game(q)`： `GET /api/search`，按游戏名搜索基础信息和攻略视频。
- `get_meta()`： `GET /api/meta`，返回定时任务是否运行和下次执行时间。
- `refresh_now()`： `POST /api/refresh`，手动触发一次完整爬取。

调用链：

```
浏览器请求 /api/refresh
-> main.refresh_now()
-> CrawlService.refresh_all()
```

app/services.py

业务服务层，连接 API、爬虫、cleaned JSON 和数据库。

主要类：

- `CrawlService`

主要函数：

- `CrawlService.refresh_all()`：完整刷新入口；使用线程锁防止重复爬取。
- `CrawlService._refresh_dashboard_tables()`：读取 cleaned JSON，将数据写入 SQLite。
- `find_strategy_videos(game_name)`：调用 B站攻略视频爬虫，返回视频列表。
- `_top_live_room_per_area(records)`：从 350 条直播房间中按分区保留热度最高的房间，用于首页展示。

调用链：

```
refresh_all()
-> batch_crawl.refresh_batch_sources()
-> _refresh_dashboard_tables()
    -> batch_crawl.load_cleaned_dataset()
    -> repository.replace_game_catalog()
    -> repository.replace_bilibili_live()
```

app/batch_crawl.py

批量采集控制器，负责一次性抓取三类数据源并导出 JSON。

主要常量：

- `DATASET_SPECS`：三类数据源的名称、展示标签、主键字段和必填字段。

主要函数：

- `refresh_batch_sources(limit=350)`：依次执行静态网页、API、动态页面三个爬虫；写入 raw 和 cleaned 文件。
- `get_batch_status()`：读取 cleaned 文件，统计数据条数和更新时间。
- `load_cleaned_dataset(name)`：读取指定 cleaned JSON，供入库使用。
- `_fetch_static()`：调用 `static_game_spider.fetch_static_games()`。
- `_fetch_api()`：调用 `bilibili_live_api_spider.fetch_live_rankings()`。
- `_fetch_dynamic()`：调用 `bilibili_dynamic_live_spider.fetch_dynamic_live_rooms()`。
- `_clean_static()`：调用 `cleaning.filter_static_games()`。

调用链：

```
refresh_batch_sources()
-> _fetch_static() -> static_game_spider.fetch_static_games()
-> _fetch_api() -> bilibili_live_api_spider.fetch_live_rankings()
-> _fetch_dynamic() -> bilibili_dynamic_live_spider.fetch_dynamic_live_rooms()
-> cleaning 过滤去重
-> _write_json()
```

app/repository.py

数据库读写和搜索诊断逻辑。

主要函数：

- `begin_run(source)`：记录一次爬取任务开始。
- `finish_run(run_id, status, record_count, error_message)`：记录爬取任务结束状态。
- `replace_game_catalog(records)`：清空并写入当前游戏目录表。
- `replace_bilibili_live(records)`：清空并写入当前 B站热度表，同时追加历史表。
- `fetch_dashboard_data()`：聚合首页需要的数据，包括游戏列表、B站热度、任务状态和指标。
- `search_game_by_name(query)`：对输入游戏名做模糊匹配，并返回诊断结果。

- `build_player_diagnosis(game, live_area)`：根据游戏排名和 B站热度输出诊断文本。
- `_match_area(game_name, bilibili_areas)`：用归一化名称匹配 B站分区。
- `_shape_game(row)`：把 SQLite 行转换成前端需要的游戏对象。

搜索调用链：

```
search_game_by_name()
-> utils.normalize_name()
-> SequenceMatcher 模糊匹配 game_catalog_current
-> _match_area()
-> build_player_diagnosis()
```

app/db.py

SQLite 建表和迁移。

主要函数：

- `ensure_data_dir()`：确保 `data/` 目录存在。
- `get_connection()`：创建 SQLite 连接。
- `connect()`：数据库上下文管理器，自动提交和关闭。
- `init_db()`：创建项目需要的三张表。
- `_ensure_column()`：给旧表补列。
- `_migrate_legacy_game_catalog()`：把旧表 `steam_popular_current` 迁移到新表 `game_catalog_current`，然后删除旧表。

当前表：

- `game_catalog_current`：首页游戏目录。
- `bilibili_live_current`：当前 B站直播热度。
- `bilibili_live_history`：B站直播热度历史。
- `crawl_runs`：爬取任务运行记录。

app/cleaning.py

清洗、过滤、去重工具。

主要函数：

- `clean_text(value)`：去掉多余空格。
- `is_valid_text(value)`：判断文本是否有效。

- `clean_url(value)`：修正 `//` 开头的链接。
- `dedupe_records(records, key_fields, required_fields, limit)`：通用去重和必填字段过滤。
- `filter_game_catalog(records)`：过滤游戏目录数据。
- `filter_static_games(records)`：过滤静态网页游戏条目和已知非游戏页。
- `filter_videos(records)`：过滤攻略视频数据。
- `filter_bilibili_live(records)`：过滤 B站直播房间数据。

app/scheduler.py

定时任务配置。

主要函数：

- `create_scheduler(crawl_service)`：创建 APScheduler，每 `REFRESH_INTERVAL_MINUTES` 分钟执行一次 `crawl_service.refresh_all()`。

app/config.py

集中配置。

重要配置：

- `DEFAULT_TARGET_RECORDS = 350`：每个数据源目标爬取条数。
- `REFRESH_INTERVAL_MINUTES = 30`：自动爬取间隔。
- `SOURCE_TIMEOUT_SECONDS = 25`：外部请求超时。
- `GAME_PARENT_AREA_IDS = {2, 3, 6}`：B站游戏相关父分区。
- `NAME_ALIASES`：游戏名别名映射，用于搜索匹配。

app/utils.py

小工具函数。

- `utc_now_iso()`：生成 UTC ISO 时间。
- `normalize_name(name)`：去掉符号、转小写、应用别名表。

3. spiders 爬虫文件

spiders/static_game_spider.py

静态网页源爬虫。

主要函数：

- `fetch_static_games(limit)`：抓取游民星空，不足时补抓 Steam 商店热销、新品、折扣等列表。
- `main()`：命令行入口，导出 raw 和 cleaned JSON。
- `_safe_fetch_gamersky()`：游民星空请求失败时返回空列表，让 Steam 补源继续执行。
- `_fetch_gamersky()`：请求游民星空游戏库并用 BeautifulSoup 解析。
- `_fetch_steam_games(seed_records, limit)`：按多个 Steam 商店列表补齐游戏目录，并交给清洗逻辑去重。
- `_fetch_steam_search_page(session, search, start)`：请求 Steam 搜索 JSON 接口。
- `_parse_steam_results(results_html)`：解析 Steam 返回的结果 HTML，提取 AppID、名称、封面和商店链接。
- `_record()`：统一静态游戏记录字段。

运行：

```
python -m spiders.static_game_spider --limit 350
python spiders/static_game_spider.py --limit 350
```

`spiders/bilibili_live_api_spider.py`

B站 API 接口源爬虫。

主要函数：

- `fetch_live_rankings(limit_per_area, limit, workers)`：并发抓取多个 B站游戏分区房间。
- `main()`：命令行入口。
- `_session()`：创建带请求头的 requests Session。
- `_fetch_room_page()`：请求单个分区的房间列表。

运行：

```
python -m spiders.bilibili_live_api_spider --limit 350 --workers 16
```

```
python spiders/bilibili_live_api_spider.py --limit 350 --workers 16
```

`spiders/bilibili_dynamic_live_spider.py`

动态页面数据源爬虫。

主要函数：

- `fetch_dynamic_live_rooms(limit, workers)`：抓取 B站直播页面背后的动态加载数据。
- `main()`：命令行入口。

说明：B站直播页面数据本质由前端动态接口加载，本项目直接抓取该动态加载接口，速度比浏览器渲染更快。

运行：

```
python -m spiders.bilibili_dynamic_live_spider --limit 350 --workers 16
python spiders/bilibili_dynamic_live_spider.py --limit 350 --workers 16
```

`spiders/bootstrap.py`

单独运行爬虫脚本时的路径引导。

主要函数：

- `ensure_project_root_on_path()`：把项目根目录加入 `sys.path`，保证 `python spiders/xxx.py` 也能导入 `app` 和 `spiders` 包。

`spiders/bilibili_strategy_video_spider.py`

攻略视频搜索爬虫。

主要函数：

- `fetch_strategy_videos(game_name, limit)`：搜索单个游戏攻略视频。
- `fetch_strategy_videos_bulk(keywords, limit, workers)`：批量搜索多个关键词。
- `_fetch_bilibili_api_page(keyword, page)`：优先请求 B站搜索接口。
- `_fetch_bilibili_dynamic_page(keyword, page_limit)`：接口不足时尝试 Playwright 动态页面抓取。

- `_fallback_search_record(keyword, reason)`：动态抓取不可用时返回 B 站搜索页入口。
- `_strip_html()`、`_first_text()`、`_first_attr()`、`_normalize_bilibili_url()`：解析辅助函数。

4. webapp 前端文件

webapp/templates/index.html

网页骨架，包含：

- 顶部标题和立即爬取按钮。
- 游戏搜索框。
- 自动定时爬取状态区域。
- 三类数据源数量卡片。
- 热门游戏卡片区。
- B 站直播热度区。
- 数据源运行状态区。

webapp/static/app.js

前端交互和渲染。

主要函数：

- `loadDashboard()`：请求 `/api/dashboard` 并刷新首页。
- `loadSchedulerStatus()`：请求 `/api/meta` 和 `/api/batch-status`，刷新定时任务状态。
- `triggerRefresh()`：点击“立即爬取”后调用 `/api/refresh`。
- `searchGame(gameName)`：调用 `/api/search`，显示搜索结果。
- `renderMetrics()`、`renderBatchStatus()`、`renderHotGames()`、`renderBilibili()`、`renderRuns()`、`renderResult()`：各区域渲染函数。
- `escapeHtml()`：避免接口数据直接插入 HTML。
- `formatNumber()`、`formatDate()`：数字和时间格式化。

webapp/static/styles.css

页面样式：

- 响应式布局。
- 卡片、按钮、搜索框、状态标签。
- 游戏卡片、B站热度条、攻略视频卡片。

5. tests 测试文件

tests/test_cleaning.py

测试清洗逻辑：

- 游戏目录去重和脏数据过滤。
- 视频链接协议补全和去重。
- 静态网页源已知非游戏页过滤。

tests/test_matching.py

测试搜索辅助逻辑：

- 游戏名别名归一化。
- 热门游戏诊断结果。

运行：

```
python -m unittest discover -s tests
```